

范式总览	Paradigms
Vibe Coding	凭感觉描述,边玩边迭代;原型/MVP 神器
Spec-Driven	先写规格(PRD/设计/AC),AI 按 spec 实现
TDD with AI	人写测试 / AI 写实现;或 AI 写双方人 review
Pair Programming	人主导,AI 副驾;⌘K 行内/Tab 补全
Autonomous Agent	Agent 自治执行;Composer/Codex exec/Background
YOLO Mode	无审批自跑,只在沙箱/throwaway 环境用
何时用哪种	
原型 → Vibe;新功能 → Spec / TDD;改老代码 → Pair;批量改动 → Agent;越关键越少自治	
心法 3 条	
▶ 一次只改 一件事,小步快跑	
▶ 看 diff 再 apply,别让 AI 静默改	
▶ 不懂的代码 别 merge,要么追问要么自己读	

AI 代码审查	Review
PR 级	
Cursor Bug Bot	挂 GitHub,自动评论
CodeRabbit	老牌 AI reviewer
Greptile	仓库级理解
Diamond / Qodo	企业级
GitHub Copilot Review	官方集成
本地	
/review	Claude Code/Codex 内建
claude -p "review HE"	脚本化
关注点	
▶ 逻辑 bug / 边界遗漏 / 错误吞掉	
▶ 安全:注入 / 路径穿越 / 密钥泄露	
▶ 性能:N+1 / 不必要 await / 阻塞 IO	
▶ 风格 / 命名 / 重复 / 死代码	
▶ 测试覆盖 / 边界用例缺失	
提示	
AI review 是 第一道筛,不能替代人;关键模块两遍 review	

Spec-Driven	规格驱动
spec 模板要素	
目标 / Why	业务背景、动机
用户故事	作为 X 我想 Y 以 Z
验收标准 AC	Given/When/Then 列表
设计 / API	数据结构 / 接口 / 错误码
边界条件	异常 / 性能 / 权限
测试用例	happy + edge + 失败路径
非目标	明确不做什么
流程	
① 写 spec → ② AI 出方案(质疑 spec)→ ③ 改 spec → ④ AI 实现 → ⑤ AI 自查 → ⑥ 人 review	
工具	
specs/ 目录	每特性一个 md
Kiro IDE	原生 spec 工作流
GitHub Spec Kit	spec-driven 套件

调试技巧	Debugging
给 AI 喂的料	
▶ 完整堆栈 + 关键变量值	
▶ 最小可复现:能 run 的 snippet	
▶ 说现象 不说原因(别误导)	
▶ 预期 vs 实际 各一句	
▶ 附近的 git log / blame	
姿势	
@错误日志	直接贴日志输出
@Lint Errors	Cursor 自动收集
@Terminal	引用刚才的命令输出
截图粘贴	UI bug 一图胜千言
反模式	
让 AI 改"它以为"是 bug 但其实是需求的代码;限定改动范围 比给上下文更重要	
死循环出口	
AI 越改越乱 → git restore . 回原点,换思路重新描述;Esc 中断 + 重 prompt	

Context Engineering	上下文工程
项目记忆文件	
CLAUDE.md	Claude Code,项目根 + 子目录
AGENTS.md	Codex / Cursor 共用
.cursor/rules/*.mdc	按 globs 自动加载
.clinerules	Cline 项目规则
必写内容	
▶ 技术栈 + 构建/测试/lint 命令	
▶ 目录职责 + 命名约定	
▶ 禁忌:别动的文件、别用的方案	
▶ 已知陷阱:历史 bug / 易踩坑	
▶ 外部资源:文档/Issue/Slack	
反模式	
塞太多无关内容、写得像营销文案、用 emoji 装饰、不更新 = 越用越歪	
不进 git 又给 AI 看	
加 .git/info/exclude 或全局	
gitignore(core.excludesfile);文件留在工作区,AI 仍可读	

Workflow 自动化	Automation
Slash Commands	
.claude/commands/*.md	项目级自定义命令
~/ .claude/commands/	全局命令
\$ARGUMENTS	命令传参占位
Hooks(harness 执行)	
PreToolUse	工具调用前拦截/审批
PostToolUse	工具后自动 lint/test
Stop / SubagentStop	完成后触发
UserPromptSubmit	prompt 前注入上下文
子 Agent / Skills	
sub-agent	隔离上下文 · 并行
skills	按需加载的工作流
CI 集成	
codex exec -a never -非交互-only	
claude -p "... " --out 脚本化 format json	
Background Agent	云端长任务

工具组合	Stack
三条主线	
CLI 系	Claude Code / Codex CLI / Aider
IDE 系	Cursor / Windsurf / Zed AI
插件系	Cline / Continue / Copilot
场景分配	
大重构 / 跨文件	Composer · Agent
局部改一处	⌘K · 行内
问答 / 解释	Chat · @file
长任务 / 后台	Background Agent
PR review	Bug Bot · CodeRabbit
CI / 脚本	codex exec · claude -p
模型选择	
规划/重构用 Opus / GPT-5 high;批量改/补全用 Sonnet / Haiku / Codex-mini 省钱	
混搭	
终端 + IDE 同开:CLI 跑长任务,IDE 看 diff 微调	

质量保障 & Pro Tips	Quality & Tips
必备守门	
▶ 类型检查:mypy/pyright/tsc 全开	
▶ linter + formatter:ruff/eslint/biome,pre-commit 兜底	
▶ 自动测试:AI 写完先跑全套	
▶ AI 输出过一遍 lint/format 再 commit	
▶ 给 AI 跑 lint 的能力(@Lint Errors /PostToolUse hook)	
Pro Tips	
▶ 一次改一件事,小步 commit,易回滚	
▶ 看 diff 再 apply,Agent 也要 review	
▶ 不懂的代码 别 merge,先问明白	
▶ AI 写的 test 要 单独 review(可能造假绿)	
▶ prompt 模板存 Notepad / commands/,复用	
▶ 大改前先 git stash / 新分支,留后悔药	
▶ token 紧 → 精确 @Files,关掉无关 @Codebase	
▶ 长任务交给 Background Agent,别盯屏幕	
▶ privacy 敏感项目开 Privacy Mode,或自带 key	